Electricity Bill Dispute Helper — Project Plan

Mini Project 11: Electricity Tariff and Billing Dispute Triage Service · UE24AM342AA5 · Evaluation 1

Team: Dhanush S (PES1UG24AM360) · Vidit Soni (PES1UG24AM318)

In short. A consumer says *"my electricity bill is wrong"*. Our service reads the Karnataka electricity rules (KERC), works out which tariff category the consumer should be in, and tells the grievance officer what to do next — quoting the exact clauses. If the rules do not clearly settle it, the system says so and hands the case to a person instead of guessing.
Why it is hard. The rule that decides a case is often printed far away from the rule a search finds first. So we use AI agents that search, ask "does anything change this?", search again and check their own answer.
What it never does. Change or recalculate a bill. The officer always makes the final decision.

1. The problem

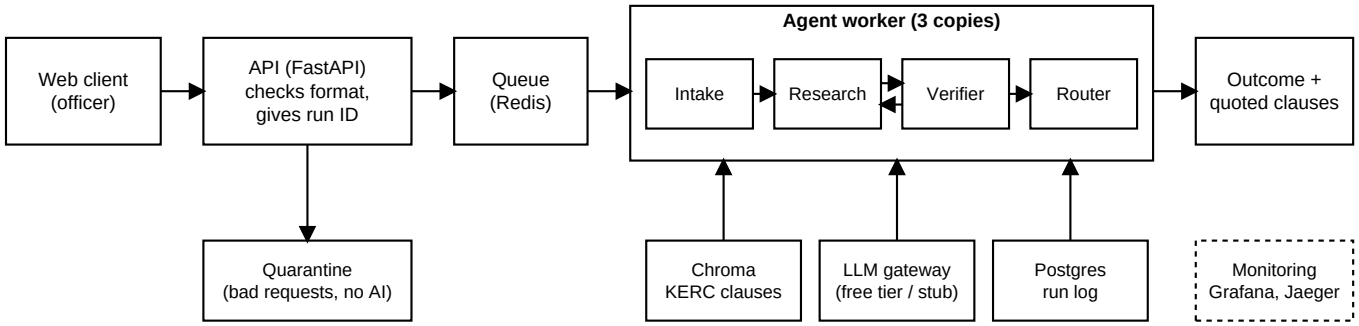
- Electricity companies get billing disputes every day. A common cause is the **tariff category** the consumer is billed under (domestic, commercial, industrial ...).
- The category is fixed by KERC's tariff order and depends on **what the premises is used for**, the **connected load**, and sometimes the time of day. A house partly used for business may be re-categorised.
- The officer must find the right category, check whether any change of category was valid, and give a **cited reason**, because the consumer can appeal to the Consumer Grievance Redressal Forum.

2. Worked example: the test case for our demo

"I have a domestic connection. I run a small tailoring shop from one room of the house. Which tariff category should I be billed under?"

A simple search (one lookup)	Our agent
Finds the domestic category (LT-2(a)) definition, because it matches the words in the question, and answers "domestic". It never sees the rule on premises used partly for business, because that rule uses different words and sits in another part of the order or in the supply code. **Confident but possibly wrong.**	1. Finds the domestic definition. 2. Asks: *is there a rule that changes this?* (mixed use, change of use, load limits) 3. Searches again with new words taken from what it just read, and finds the mixed-use rule. 4. Checks that both quoted clauses really appear in the text it found. 5. Gives an outcome citing **both** clauses, or escalates if together they still do not settle it.

3. How the system works



- **Front door:** every request is checked against a published format (fields, ranges, allowed values). Bad requests are stored in quarantine with the reason, and no AI runs on them.
- **Run ID:** a case takes about 30 seconds, so the API replies at once with a run ID and the web page checks back for the result. A queue sits between the API and the workers.
- **Four agent roles: Intake** works out the dispute type and pulls out the facts (use of premises, load, billed category). **Research** searches the documents using tools (search, fetch clause by number, read the request's fields). **Verifier** checks every citation against the text actually found and sends the case back for one more search, or escalates. **Router** picks the outcome and blocks anything that looks like a bill change.
- **Safety limits:** at most 12 steps, 10 tool calls and 90 seconds per case. If a limit is hit, or the AI provider is down, the case is escalated, never answered half-done.

Outcome	When the system gives it
Bill stands (with reason)	The cited clauses confirm the category the consumer is billed under
Revision recommended	The cited clauses point to a different category
Site inspection required	The answer depends on facts that must be checked on site (actual use, load)
Escalate to officer	Rules are mixed or conflict, the case is outside our documents, or the answer could not be verified

4. Documents we use (Karnataka, KERC)

- **KERC tariff order** currently in force for BESCOM — definitions of each category (LT-2(a) domestic, LT-3 commercial, LT-5 industrial) and the notes under them.
- **KERC Conditions of Supply** (Karnataka's supply code) — rules on change of use, re-categorisation and inspection.
- Both are free PDFs. We split them clause by clause and keep the notes under each category as separate pieces, since that is where mixed-use rules usually sit. Embeddings are computed locally, so indexing costs nothing. At Evaluation 1 we will show the exact passages the tailoring case depends on.

5. Tools

Need	Tool	Need	Tool
Agents	LangGraph	API	FastAPI
Document search	LangChain + Chroma, all-MiniLM-L6-v2	Queue and cache	Redis
Run log, feedback	PostgreSQL	Run everything	Docker Compose (one command)
Monitoring	Prometheus, Grafana, Jaeger	Testing	TruLens (quality), Locust (load)
Language model	Free-tier provider behind our own gateway, switchable by config	Load-test model	Stub that replies after a fixed delay

6. How we will show it works

- **Answer quality:** 20+ test cases, each with the expected outcome and the clause that supports it (including the tailoring case). We measure groundedness, context relevance, faithfulness and hallucination rate, and compare simple search vs our agent, and two prompt versions.
- **Speed target (fixed now, before testing):** 95% of cases finish within **60 seconds** with 10 cases running at once. The officer reads the result while the case file is open, so about a minute is the useful limit. Load tests use the stub model so we measure our system, not the provider's limits.
- **Safety:** checks for prompt injection and toxic text, no unverified answer is ever returned, and a circuit breaker switches to a fallback when the provider fails.
- **Traceability:** from a run ID we can replay every search, passage, prompt version, step time and cost.

7. Timeline and roles

Week	Goal
1	Download documents, locate the tailoring-case passages, define request format, set up repo and containers
2	**First full working path:** request → one agent → answer with citations, shown in the web page
3	Research and Verifier agents, the four outcomes, escalation, safety checks
4	Timeouts, retries and circuit breaker; monitoring dashboards; 20+ test cases
5	Prompt comparison, load test and scaling to 3 workers, caching, cost per 100 cases
6	Runbook, incident log, README tested from a clean clone, report, demo practice

Member	Responsible for
Dhanush S	Documents and indexing, the agents and their tools, test cases and quality metrics, safety checks
Vidit Soni	API and request format, Docker setup and model gateway, monitoring, load testing, caching, cost tracking
Both	Web page, runbook and incident log, report, demo

Main risks: the mixed-use rule may be unclear (then escalation is the correct answer); free-tier rate limits (handled by caching, per-user limits and the stub); scanned PDF pages (OCR fallback).